

Gated Recurrent Units (GRUs).

1. Introduction and Comparison to LSTM

The first page establishes what a GRU is by comparing it to an LSTM:

- **LSTMs** manage both long-term and short-term memory through separate cell states and hidden states.
- **GRUs** are described as an "updated version of LSTM" that simplifies this by combining the long-term and short-term memory into a single hidden state.

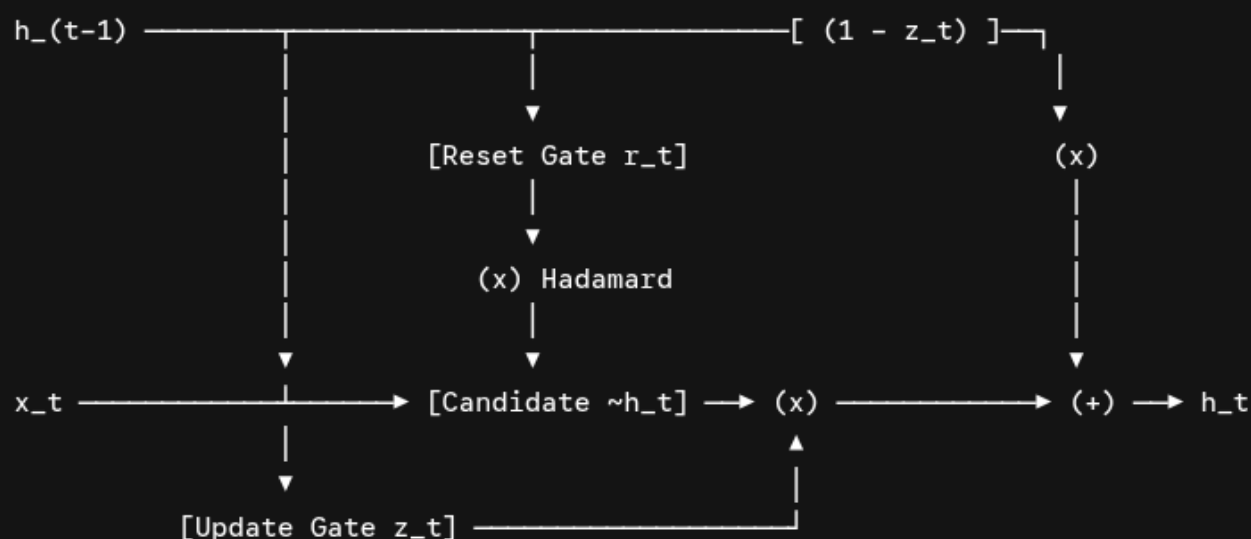
2. High-Level Architecture

The first block diagram illustrates the core mechanism of a GRU cell.

- It takes two inputs: the previous hidden state $h^{(t-1)}$ and the current input data $x^{(t)}$.
- These inputs are passed into two primary control mechanisms: the **Update Gate** and the **Reset Gate**.
- The cell outputs the new hidden state $h^{(t)}$ and the predicted output $y^{(t)}$.

Core Architecture & Dataflow :

A **Gated Recurrent Unit (GRU)** is a gating mechanism for recurrent neural networks designed to solve the vanishing gradient problem while maintaining a more lightweight profile than a standard Long Short-Term Memory (LSTM) network. While an LSTM relies on three gates (input, forget, output) and maintains two separate states (cell state and hidden state), a GRU merges these into a single hidden state h_t controlled by just **two gates: the Reset Gate and the Update Gate**.



The Two Gates Explained

Unlike standard LSTMs, GRUs do not have a dedicated "forget gate." Instead, the forgetting and updating responsibilities are distributed across two gates:

- **Reset Gate (r_t):** Dictates how much of the previous hidden state h_{t-1} to disregard when computing new candidate information. If $r_t \approx 0$, the unit discards past memory and acts like it is reading the first symbol of a sequence.
- **Update Gate (z_t):** Acts simultaneously as both a forget gate and an input gate. It decides how much of the past memory h_{t-1} should be preserved versus how much of the new candidate memory $\tilde{h}_t$ should be written into the new state h_t .

Mathematical Equations & Step-by-Step Terms

The internal computation follows four sequential steps:

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r)$$

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z)$$

$$\tilde{h}_t = \tanh(W_h x_t + U_h(r_t \odot h_{t-1}) + b_h)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

- x_t and h_{t-1} : The current input vector at time step t and the hidden state from time step $t - 1$.
- W and U : Learnable weight matrices mapping inputs and recurrent hidden states to gate activations.
- σ and $\tanh$: The Sigmoid function (squashing values to $[0, 1]$ for gating) and Hyperbolic Tangent (squashing candidate values to $[-1, 1]$).
- $\odot$: The Hadamard (element-wise) product, which scales individual feature activations independently.

Why GRUs Remain Popular

- **Computational Efficiency:** Having approximately 25% fewer parameters than an LSTM results in faster training epochs and lower GPU memory overhead.
- **Performance on Smaller Datasets:** With fewer parameters to tune, GRUs are less prone to overfitting when sequence datasets are limited or moderately sized.
- **Embedded and Edge Deployments:** Their compact memory footprint makes them suitable for real-time inference on edge microcontrollers and low-power robotic systems.